

Force-Directed List-Scheduling for High-Level Synthesis

Faysal Ahammed Tonmoy
Student ID: 1230473
HW/SW Codesign Seminar Paper

Abstract—High-level synthesis (HLS) translates a behavioral description into a register-transfer level implementation by deciding which operations are executed in which control steps and on which hardware resources. Scheduling is therefore a central optimization problem because it directly affects latency, the number of functional units, register pressure, multiplexing, and the final control structure. This paper studies force-directed list-scheduling (FDLS) as a constructive heuristic for resource-aware HLS scheduling. The method combines the implementation simplicity of list scheduling with the global balancing idea of force-directed scheduling. Instead of using a fixed priority order, it repeatedly estimates operation mobility, operation probabilities, and resource-type distributions, then uses force values to guide the selection of ready operations. The paper explains the mathematical model, algorithmic structure, runtime implications, and a compact C++ implementation test. It also evaluates the method against list scheduling and integer linear programming, with attention to embedded and FPGA-oriented design contexts.

Index Terms—High-level synthesis, force-directed scheduling, list scheduling, resource constraints, RTL, CDFG.

I. INTRODUCTION

Modern embedded systems often require hardware accelerators for signal processing, control, communication, and sensor processing. Manual register-transfer level (RTL) design is powerful but time-consuming; high-level synthesis addresses this gap by transforming an algorithmic or behavioral description into a structural hardware implementation. In the standard HLS flow, the designer starts from a behavioral model, often represented internally as a control/data-flow graph (CDFG), and the synthesis tool derives datapath resources, storage elements, interconnect, and a controller. De Micheli's architectural-level synthesis view separates the temporal problem of scheduling from spatial problems such as binding and resource sharing [3]. Teich and Haubelt similarly describe synthesis through allocation, scheduling, and binding across hardware/software abstraction levels [2]. Gajski et al. provide a broader introduction to high-level synthesis as the transformation from behavioral descriptions to chip and system implementations [4].

Scheduling is difficult because it is not only a question of ordering operations. A good schedule must respect data dependencies, resource capacities, operation delays, and the latency target. For example, starting all possible operations as soon as possible may reduce latency but increase the required number of multipliers and adders. Conversely, spreading operations

may reduce area but can increase the number of control steps. This is a core area/latency tradeoff of HLS.

Force-directed scheduling (FDS), introduced by Paulin and Knight, addresses this tradeoff by using distribution and force concepts to guide scheduling decisions [1]. In the formulation used in this paper, force-directed list-scheduling combines this force idea with the constructive time-step structure of list scheduling. It is not an exact optimization method. Rather, it is a practical heuristic: it seeks a valid schedule quickly while using a more global priority estimate than classical list scheduling.

II. HLS SCHEDULING PROBLEM

A. Graph and resource model

Let the behavioral specification be represented as a directed acyclic graph

$$G = (V, E), \quad (1)$$

where each node $v_i \in V$ is an operation and each edge $(v_i, v_j) \in E$ means that v_j can start only after v_i has produced its result. Each operation has a delay d_i in control steps and belongs to one resource type, such as multiplier, adder/subtractor, comparator, or memory port. Let R be the set of resource types. The mapping

$$\beta(v_i) = r_k \quad (2)$$

assigns operation v_i to a compatible resource type r_k . The allocation function

$$\alpha(r_k) \in \mathbb{N} \quad (3)$$

gives the number of available instances of resource type r_k . A schedule is a start-time function $\tau : V \rightarrow \mathbb{N}_0$.

A valid resource-constrained schedule must satisfy two classes of constraints. First, precedence constraints require

$$\tau(v_j) \geq \tau(v_i) + d_i \quad \forall (v_i, v_j) \in E. \quad (4)$$

Second, at every control step t , the number of operations using a resource type must not exceed the allocation:

$$|\{v_i \mid \beta(v_i) = r_k, \tau(v_i) \leq t < \tau(v_i) + d_i\}| \leq \alpha(r_k). \quad (5)$$

The output of this step contributes to the final RTL structure: scheduled operations become control-step actions, the allocated functional units become datapath resources, and the controller activates the correct transfers in each state.

B. ASAP, ALAP, and mobility

The earliest feasible start time of an operation is obtained by as-soon-as-possible (ASAP) scheduling, while the latest feasible start time under a latency limit $\bar{L}$ is obtained by as-late-as-possible (ALAP) scheduling. The mobility interval of operation v_i is written as $[\tau_i^S, \tau_i^L]$, where τ_i^S and τ_i^L are the earliest and latest feasible start steps. The mobility is

$$\mu_i = \tau_i^L - \tau_i^S. \quad (6)$$

A zero-mobility operation has no scheduling freedom and is normally critical. A high-mobility operation can be placed in several control steps and is useful for balancing resource usage.

III. FROM LIST SCHEDULING TO FDLS

A. Classical list scheduling

List scheduling is a constructive heuristic. It scans time steps and schedules operations that are ready, meaning all predecessors have completed. The key design choice is the priority function $p(v_i)$. Common priorities include longest path to output, number of successors, and mobility. Teich and Haubelt describe list scheduling as a development of ASAP scheduling that uses global priority criteria, while still constructing the schedule control step by control step [2].

For a resource type r_k and time t , the ready candidate set can be described as all unscheduled operations of type r_k whose predecessors have already completed by time t . List scheduling chooses the highest-priority candidates while respecting $\alpha(r_k)$. Its strength is speed and simplicity. Its weakness is that the priority is usually computed before scheduling starts. Therefore, the priority may become less suitable after earlier decisions have changed the remaining design freedom.

B. Force idea

FDS changes the priority concept. Instead of assigning one static priority, it estimates how the placement of an operation changes the distribution of resource usage over time. For each unscheduled operation v_i , the probability of starting at a feasible time s is approximated by

$$p_{i,s} = \frac{1}{\mu_i + 1} \quad \text{for } s \in [\tau_i^S, \tau_i^L], \quad (7)$$

and $p_{i,s} = 0$ outside this interval. For unit-delay operations, the resource distribution for type r_k is

$$q_{k,t} = \sum_{i: \beta(v_i)=r_k} p_{i,t}. \quad (8)$$

For multi-cycle operations with delay d_i , the resource is occupied during the full interval from s to $s+d_i-1$. Therefore, the occupied-resource distribution can be written as

$$q_{k,t} = \sum_{i: \beta(v_i)=r_k} \sum_{s=\max(\tau_i^S, t-d_i+1)}^{\min(\tau_i^L, t)} p_{i,s}. \quad (9)$$

A high $q_{k,t}$ means that many operations are expected to compete for the same type of resource in control step t . Placing an operation in a crowded step can increase resource pressure, while placing it in a less crowded step can flatten the distribution.

The self-force of placing v_i in step t can be interpreted as the change in the distribution of its own resource type:

$$F_{i,t}^S = \sum_m q_{k,m} \Delta p_{i,m}. \quad (10)$$

Here, $\Delta p_{i,m}$ is the probability change caused by fixing v_i to step t . Because fixing v_i can also restrict the mobility of predecessors and successors, predecessor and successor forces are added:

$$F_{i,t} = F_{i,t}^S + \sum_{v_j \in \text{pred}(v_i)} F_{j,t}^N + \sum_{v_j \in \text{succ}(v_i)} F_{j,t}^N. \quad (11)$$

In latency-constrained FDS, the operation-time pair with the lowest force is selected to reduce expected resource peaks. In force-directed list-scheduling, the scheduler keeps the list-scheduling structure but updates mobility intervals and force values dynamically. Among ready operations, the candidate with the lowest force cost is preferred under the given resource constraints. Thus, FDLS is a resource-constrained constructive heuristic, while classical FDS is usually presented as a latency-constrained resource-minimization heuristic.

C. Algorithmic structure

The following pseudocode expresses FDLS as a ready-list scheduler with dynamic force-based priorities.

Listing 1. Core FDLS idea in C++-like pseudocode.

```

1 while (!all_scheduled()) {
2   update_ASAP_ALAP_for_unscheduled_nodes();
3   update_operation_probabilities();
4   update_resource_distributions();
5
6   for (time t = current_step; has_ready_work(t); ++t) {
7     for (resource_type r : resources) {
8       auto ready = ready_nodes_of_type(r, t);
9
10      for (auto v : ready)
11        priority[v] = force_based_priority(v, t);
12
13      schedule_best_nodes(ready, alpha[r]);
14    }
15  }
16 }
```

The implementation differs from exact mathematical programming in one important way: it never solves the complete search problem. It commits decisions step by step. However, it repeatedly refreshes the force estimates, so the priority reflects the current partial schedule.

D. Detailed decision cycle

The FDLS decision cycle can be explained as five repeated phases. First, the scheduler computes the current feasible window of every unscheduled operation. This window is not identical to the initial ASAP/ALAP window because already fixed operations may tighten predecessor or successor constraints. Second, the scheduler converts each remaining window into

an execution probability using (7). Third, the probabilities are accumulated per resource type using (8) or (9). Fourth, every feasible ready-operation placement is evaluated by the force equation. Fifth, the scheduler commits one or several ready operations, depending on the available number of resource instances.

This cycle is important because it separates feasibility from preference. Feasibility is decided by dependency and resource constraints. Preference is decided by force. Therefore, FDLS does not schedule an operation merely because it has a low force if the operation is not ready or if the resource capacity is already exhausted. This makes the method safe as a constructive scheduler: every committed decision respects the local constraints at the moment it is made. The quality question is different: whether the committed sequence is globally best. As with most heuristics, the answer is not guaranteed.

For example, if v_1 has window $[0, 0]$, v_2 has window $[0, 1]$, and v_3 has window $[1, 2]$, then a unit-delay probability model gives $p_{1,0} = 1$, $p_{2,0} = p_{2,1} = 0.5$, and $p_{3,1} = p_{3,2} = 0.5$. For one resource type, the resulting distribution is $q_{k,0} = 1.5$, $q_{k,1} = 1.0$, and $q_{k,2} = 0.5$. This means step 0 is the most crowded step for that resource type.

IV. APPLICATION EXAMPLE

Consider a simple HLS problem containing two multiplications and two additions feeding a final addition. Suppose one multiplier and one ALU are available. A pure ASAP schedule would start both multiplications at the first step if there were two multipliers. With only one multiplier, a scheduler must decide which multiplication goes first. A static list scheduler may use longest path or node index as a tie breaker. FDLS instead checks how the choice changes the remaining probability distribution.

One valid illustrative schedule is the following: step 0 executes v_1 on the multiplier; step 1 executes v_2 on the multiplier and v_3 on the ALU; step 2 executes v_4 on the ALU; and step 3 executes v_5 on the ALU. The important point is not that this small example proves optimality. Rather, it shows the type of decision FDLS makes: if scheduling v_1 first creates a more balanced successor window than scheduling v_2 first, the force measure prefers v_1 . In a larger datapath, such balancing can reduce peak resource pressure and avoid early choices that later create bottlenecks.

This is also why FDLS is relevant to practical HLS. A final RTL design is not only a timeline. It creates hardware: functional units, registers for live values, multiplexers around shared units, and controller states. If the schedule creates unnecessary parallelism, the binding stage may require more units or more interconnect. If it serializes too aggressively, latency can grow. FDLS tries to keep this tradeoff visible during scheduling.

V. RUNTIME, MEMORY, AND IMPLEMENTATION ASPECTS

The runtime of a basic list scheduler is close to linear in the number of operations when ready lists and dependency counts

are maintained efficiently. Teich and Haubelt give list scheduling as a low-overhead constructive algorithm and emphasize that it produces schedules satisfying resource constraints by construction [2]. FDLS adds overhead because ASAP/ALAP windows, probabilities, distributions, and forces are updated repeatedly. If $n = |V|$, T is the latency window, and $m = |R|$, a simple implementation can require $O(nT)$ memory for probability tables and $O(mT)$ memory for resource distributions. A straightforward recomputation of all forces after each scheduled operation can approach $O(n^2T)$ time. Practical implementations reduce the cost by updating only affected predecessors, successors, and resource-type distributions.

The memory overhead is moderate for small and medium datapaths, because the main arrays are indexed by operation and control step. The method becomes less attractive when the graph is very large, highly control-dependent, or when operation delays and resource choices are strongly heterogeneous. In those cases, exact ILP formulations may be too slow, but a more specialized heuristic or constraint-based scheduler may be preferable.

Listing 2 shows a compact C++ fragment for the probability and distribution update. It is not a complete HLS tool; it is the main feature needed to implement the force part of FDLS.

Listing 2. Probability and resource-distribution for multi-cycle operations.

```

1  struct Node {
2      int type; // resource type
3      int asap; // earliest start step
4      int alap; // latest start step
5      int delay; // execution length in control steps
6      bool fixed = false;
7  };
8
9  vector<vector<double>> p(n, vector<double>(T, 0.0));
10 vector<vector<double>> q(numTypes, vector<double>(T, 0.0))
11 ;
12
13 for (int i = 0; i < n; ++i) {
14     if (nodes[i].fixed) continue;
15
16     int mob = nodes[i].alap - nodes[i].asap;
17     double prob = 1.0 / double(mob + 1);
18
19     for (int s = nodes[i].asap; s <= nodes[i].alap; ++s) {
20         p[i][s] = prob;
21
22         for (int u = s; u < s + nodes[i].delay && u < T; ++u)
23             q[nodes[i].type][u] += prob;
24     }
25 }

```

A. Reference implementation and test result

To satisfy the implementation requirement, a small standalone C++17 program was written and executed. The program does not implement a full commercial HLS compiler. Instead, it tests the main feature of FDLS: using mobility windows, probability-based resource distributions, and force-based candidate evaluation to select ready operations. The implemented application example contains five operations: two multiplier operations, two intermediate ALU operations, and one final ALU operation. The dependency structure is $v_1 \rightarrow v_3$, $v_2 \rightarrow v_4$, and $(v_3, v_4) \rightarrow v_5$. The program computes ASAP and ALAP windows, derives mobility values, builds

```

main: force-based choice
Operation windows:
v1 type=MUL ASAP=0 ALAP=1 mobility=1
v2 type=MUL ASAP=0 ALAP=1 mobility=1
v3 type=ALU ASAP=1 ALAP=2 mobility=1
v4 type=ALU ASAP=1 ALAP=2 mobility=1
v5 type=ALU ASAP=2 ALAP=3 mobility=1

Before step 0 resource distribution q:
MUL: 1.00 1.00 0.00 0.00
ALU: 0.00 1.00 1.50 0.50
Ready MUL candidates: v1(force=0.00) v2(force=0.00)
-> schedule v1 on MUL at step 0
Ready ALU candidates: none

Before step 1 resource distribution q:
MUL: 0.50 0.50 0.00 0.00
ALU: 0.00 1.00 1.50 0.50
Ready MUL candidates: v2(force=0.00)
-> schedule v2 on MUL at step 1
Ready ALU candidates: v3(force=-0.25)
-> schedule v3 on ALU at step 1

Before step 2 resource distribution q:
MUL: 0.00 0.00 0.00 0.00
ALU: 0.00 0.50 1.00 0.50
Ready MUL candidates: none
Ready ALU candidates: v4(force=0.25)
-> schedule v4 on ALU at step 2

Before step 3 resource distribution q:
MUL: 0.00 0.00 0.00 0.00
ALU: 0.00 0.00 0.00 0.00
Ready MUL candidates: none
Ready ALU candidates: v5(force=0.00)
-> schedule v5 on ALU at step 3

Final Gantt-style schedule:
Step 0: v1(MUL)
Step 1: v2(MUL) v3(ALU)
Step 2: v4(ALU)
Step 3: v5(ALU)

... Program finished with exit code 0
Press ENTER to exit console [1]

```

Fig. 1. First part of the C++ FDLS implementation output. The output shows operation windows, ASAP/ALAP values, mobility values, and the first force-based scheduling decisions.

```

Ready ALU candidates: none

Before step 1 resource distribution q:
MUL: 0.50 0.50 0.00 0.00
ALU: 0.00 1.00 1.50 0.50
Ready MUL candidates: v2(force=0.00)
-> schedule v2 on MUL at step 1
Ready ALU candidates: v3(force=-0.25)
-> schedule v3 on ALU at step 1

Before step 2 resource distribution q:
MUL: 0.00 0.00 0.00 0.00
ALU: 0.00 0.50 1.00 0.50
Ready MUL candidates: none
Ready ALU candidates: v4(force=0.25)
-> schedule v4 on ALU at step 2

Before step 3 resource distribution q:
MUL: 0.00 0.00 0.00 0.00
ALU: 0.00 0.00 0.00 0.00
Ready MUL candidates: none
Ready ALU candidates: v5(force=0.00)
-> schedule v5 on ALU at step 3

Final Gantt-style schedule:
Step 0: v1(MUL)
Step 1: v2(MUL) v3(ALU)
Step 2: v4(ALU)
Step 3: v5(ALU)

... Program finished with exit code 0
Press ENTER to exit console [1]

```

Fig. 2. Second part of the C++ FDLS implementation output. The output shows the remaining force-based decisions and the final Gantt-style schedule.

the resource distribution graph q , prints force values for ready candidates, and finally prints a Gantt-style schedule.

The tested C++ output confirms that the scheduler respects both dependency and resource constraints. As shown in Fig. 1 and Fig. 2, the final schedule is v_1 on the multiplier in step 0, v_2 on the multiplier and v_3 on the ALU in step 1, v_4 on the ALU in step 2, and v_5 on the ALU in step 3. This demonstrates the main scheduling feature of FDLS on a compact application example.

VI. SCIENTIFIC CONTEXT AND TRADEOFFS

FDLS belongs to a family of resource-constrained scheduling heuristics for architectural synthesis. It is close to list scheduling because it constructs a schedule directly. It is close to FDS because it uses mobility intervals and distribution graphs to guide decisions. It differs from ILP scheduling, which models start-time variables and constraints explicitly and can search for an optimum under a defined objective, but at much higher computational cost. De Micheli places scheduling algorithms beside binding and resource-sharing algorithms as part of architectural-level synthesis [3]. This placement is important: a schedule that looks good alone may still produce poor binding or high interconnect cost.

The main engineering tradeoff is optimality versus feasibility. Exact optimization is attractive for small examples and for

reference solutions, but HLS tools must often process larger graphs quickly. FDLS gives no general proof of optimality, but it uses more information than a fixed list priority. Another tradeoff is latency versus area. If the target is fixed latency, force-directed decisions tend to reduce expected resource peaks. If the target is fixed allocation, list-style scheduling ensures resource feasibility while the force priority aims to minimize latency or avoid bottlenecks.

For embedded and FPGA-oriented systems, FDLS is suitable when the design is datapath-dominated and has enough mobility to exploit. Examples include arithmetic kernels, filters, transforms, and small accelerators where multipliers, ALUs, and memory ports are constrained. It is less suitable for strongly control-dominated code, pointer-heavy software, dynamic loops with unknown bounds, or cases where memory banking dominates the schedule more than arithmetic resources. It is also less suitable when the designer needs a formal optimality certificate.

A. Implementation boundary

The implementation scope is deliberately limited to the scheduling feature. It includes the CDFG/DAG representation, topological order assumption, ASAP/ALAP computation, ready-list checks, probability and distribution update, force-cost evaluation, and schedule printing. It does not include full C parsing, register allocation, binding, VHDL generation, or timing closure. This boundary is acceptable because the seminar topic is the scheduling heuristic, not a complete commercial HLS compiler.

VII. CRITICAL EVALUATION AND CONCLUSION

The strength of FDLS is that it improves the decision quality of list scheduling without moving to a full exact solver. It captures a useful intuition: an operation should not be scheduled only because it is ready; it should be scheduled when doing so preserves good resource balance for the remaining graph. This is especially helpful when several ready operations have similar local priority but very different downstream consequences.

Its limitations are equally important. The force model is an approximation. Uniform probability over a mobility interval is simple, but it does not fully represent all future dependency interactions. Considering only direct predecessor and successor effects reduces runtime, but it can miss longer-range consequences. The method also depends on the quality of ASAP/ALAP bounds and the selected latency or resource assumptions. Finally, it solves only part of HLS. The final hardware quality still depends on allocation, binding, register sharing, memory architecture, and technology mapping.

A reasonable extension is to integrate FDLS with binding feedback, so that scheduling decisions also estimate register lifetime, multiplexer cost, or memory-port contention. Another extension is to adapt the force function for energy-aware HLS, where switching activity and memory access are added to the cost. For a seminar implementation, the most useful scope is a small C++ scheduler that reads a

DAG, computes ASAP/ALAP times, updates probabilities and forces, and prints a Gantt-style schedule. This demonstrates the main technical contribution while keeping the implementation understandable.

In conclusion, force-directed list-scheduling is best understood as a practical compromise. It is more informed than simple list scheduling and more scalable than exact optimization. For high-level synthesis of resource-constrained datapaths, that compromise is often scientifically and engineering-wise meaningful.

AI USAGE AND PROTOCOL

This paper was prepared with AI assistance for topic structuring, wording improvement, consistency checking, and implementation explanation. The AI output was not used as an independent technical source. Technical claims were checked against the cited HLS literature, especially Paulin and

Knight, Teich and Haubelt, De Micheli, and Gajski et al. The detailed AI-supported workflow is documented separately in the submitted AI usage protocol JSON and BibTeX file [5].

REFERENCES

- [1] P. G. Paulin and J. P. Knight, "Force-directed scheduling for the behavioral synthesis of ASICs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 8, no. 6, pp. 661–679, Jun. 1989, doi: 10.1109/43.31522.
- [2] J. Teich and C. Haubelt, *Digitale Hardware/Software-Systeme: Synthese und Optimierung*, 2nd ed. Berlin, Germany: Springer, 2007.
- [3] G. De Micheli, *Synthesis and Optimization of Digital Circuits*. New York, NY, USA: McGraw-Hill, 1994.
- [4] D. D. Gajski, N. D. Dutt, A. C.-H. Wu, and S. Y.-L. Lin, *High-Level Synthesis: Introduction to Chip and System Design*. Norwell, MA, USA: Kluwer Academic Publishers, 1992.
- [5] F. A. Tonmoy, "AI Usage Protocol for Force-Directed List-Scheduling for High-Level Synthesis," submitted AI usage protocol notebook, JSON file, and BibTeX file, HW/SW Codesign Seminar, 2026.